Personalized Alzheimer's Disease Neural Network Model

Yilin Li
2113840@mail.nankai.edu.cn
School of Mathematical Sciences, Nankai University

2025-09-22

Abstract

In this paper, we introduce a neural network–based machine learning model for Alzheimer's Disease kinetics, focusing on illustrating the algorithm for constructing both population-level and personalized models, as well as uncertainty quantification.

Keywords Alzheimer's Disease · Dynamic System Modeling · Machine Learning · Personalized Medicine · Uncertainty Quantification

1 Introduction

Alzheimer's disease has long been one of the most threatening chronic illnesses affecting the lives of the elderly. Research on Alzheimer's disease has often focused on various biomarkers, such as the expression levels of A_β and τ proteins, and their relationship with cognitive decline. Clearly, there are inherent dynamics in this process; therefore, establishing mathematical models to accurately uncover and describe these dynamics has become a subject of both theoretical and practical importance. Zheng et al. constructed a polynomial model based on sparse identification, which consisted of quadratic polynomial equations, with the coefficients suggesting a transmission chain of $A_\beta \to \tau \to N \to C$ [1]. However, this sparse identification approach is limited to quadratic polynomials, since higher-order terms would introduce significant complexity and numerical instability. Yet, relying solely on quadratic polynomials may oversimplify the model, especially as its interpretability remains essentially at the level of logistic regression. To address this, we investigate various neural network modeling approaches, namely replacing the dynamic terms on the right-hand side of the equations with FNNs; FNNs combined with quadratic polynomials (with learnable coefficients, similarly hereafter) via residual learning; or FNNs multiplied by quadratic polynomials. By adopting ideas from PINNs, we aim to strike a balance among interpretability, model performance, and complexity. Finally, building upon the population-level model, we apply a similar approach to that of Zheng et al. and incorporate fine-tuning techniques to identify key parameters in the model. By adjusting these parameters, we achieve personalized modeling and prediction, thereby providing valuable intelligent support for the future prevention and treatment of Alzheimer's disease. In addition, we will provide uncertainty quantification of the model based on statistical methods, including estimates such as confidence intervals for the trajectories.

We study mainly 4 biomarkers related to AD, data are given by ADNI dataset: A_β, τ, N, C. The ODE model could be described as a 1-order ODE:

$$\frac{\mathrm{d}y}{\mathrm{d}s} = f(y), y = \left(A_\beta, \tau, N, C\right) \in R^4 \tag{1}$$

The polynomial (population) model proposed by Zheng et al. is as follows[1]:

$$\begin{cases} \frac{\mathrm{d}A_\beta}{\mathrm{d}s} = 0.917A_\beta - 0.873A_\beta^2 \\ \frac{\mathrm{d}\tau}{\mathrm{d}s} = 0.788\tau - 0.246\tau^2 + 0.002A_\beta + 3.066A_\beta^2 - 3.650A_\beta\tau \\ \frac{\mathrm{d}N}{\mathrm{d}s} = 1.627N - 1.253N^2 + 0.018\tau + 2.342\tau^2 - 4.015\tau N \\ \frac{\mathrm{d}C}{\mathrm{d}s} = 0.159C + 0.202C^2 + 0.010N + 0.019N^2 - 0.176NC \end{cases} \tag{2}$$

each biomarker was first normalized, i.e.

$$y = \frac{y - y_{\text{mean}}}{\sigma} \tag{3}$$

where $y_{\text{mean}} \in R^4$ denotes mean value of y and σ denotes deviation.

However, in order to place all patients on a unified temporal scale for assessing disease progression—that is, to determine the actual stage of severity—we cannot make direct comparisons on the original age scale (most individuals being between 60 and 90 years old). Instead, age must first be linearly transformed into the Disease Progression Score (DPS) scale (hereafter referred to as the DPS transform, with the resulting score denoted as DPS or simply s), upon which the model is then trained.

2 DPS Transformation

2.1 Introduction

For each patient i, we apply DPS transformation as follows:

$$s_i = a_i t_i + b_i \tag{4}$$

where a_i, b_i are patient-specific parameters and t_i denotes patient i's age series.

Unlike the approach in Zheng et al[1]., where the model parameters and the DPS transformation parameters are updated simultaneously, in this work we pre-train the DPS transformation parameters and subsequently keep them fixed during the main training process. This design choice is based on empirical observations: jointly optimizing the DPS parameters with the model often leads to numerical instability, particularly with the DPS score s frequently jumping outside the range of –60 or +600, exhibiting large and unpredictable scales, leaving a gap in interpretability. After comparing the two algorithms, we believe this difference may arise because Zheng et al.'s model is relatively simple and thus employs the more sophisticated and robust Levenberg–Marquardt algorithm, whereas our implementation relies on PyTorch with the Adam optimizer, which results in reduced numerical stability.

Here we first display the original scatter plot in which the horizontal axis is the original age axis in Figure 1.

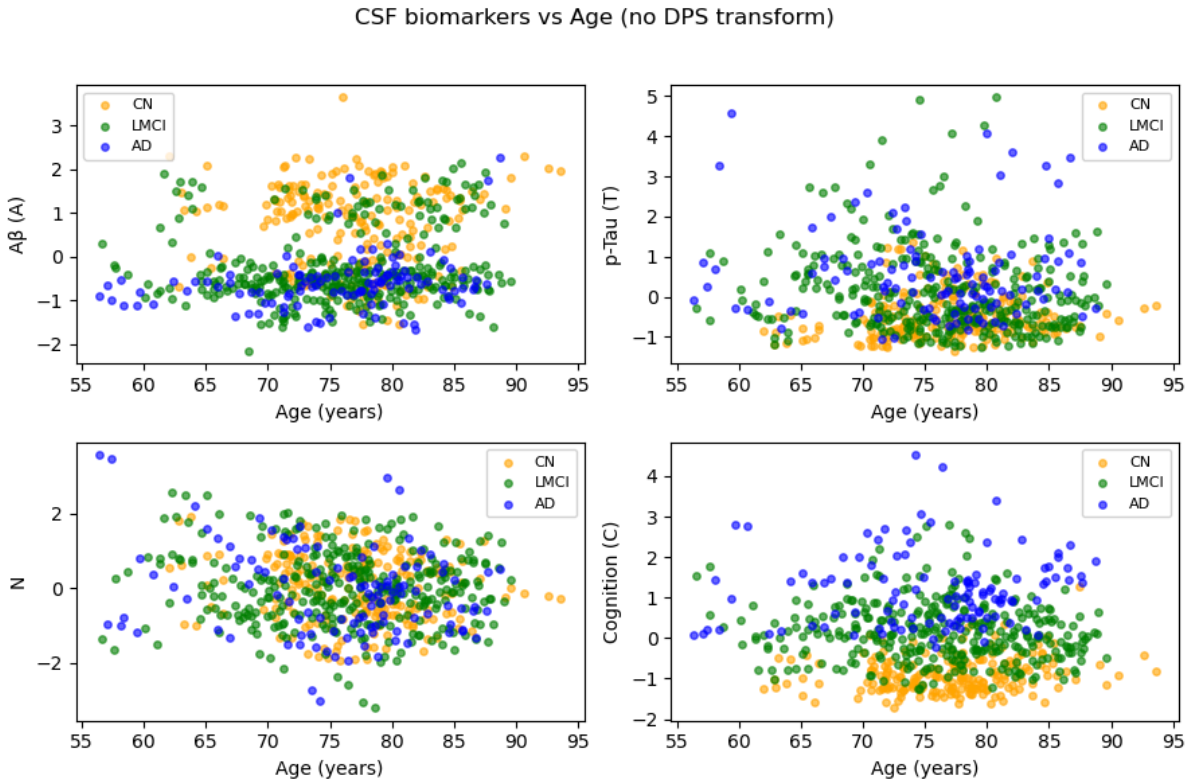

Figure 1: raw data without DPS transformation

2.2 Pretrain Stage

Directly training the polynomial model and DPS parameters introduces numerical instability: if polynomial parameters are too small, the trajectory becomes a straight line, while slightly larger values lead to numerical explosion. We therefore establish a separate pre-training process. This process fits a Sigmoid function to the data while optimizing DPS parameters, then fits a preliminary polynomial model using the Sigmoid's analytical derivative. This provides a stable basis of pre-trained DPS parameters and a polynomial model, allowing us to subsequently investigate the hybrid neural network. The specific algorithm is divided into the following three key steps:

2.2.1 Step 1: Initialization of DPS Parameters Based on Clinical Stage

To ensure the initial Disease Progression Score s (where $s_{i(t)} = a_i t + b_i$) roughly reflects the patient's clinical status, we first set an initial group of DPS parameters (a_i, b_i) for each patient based on their known diagnostic labels (CN, LMCI, AD).

- **Parameter a_i (Progression Rate)**: This parameter is set to a fixed value based on the severity of the clinical stage. Cognitively normal (CN) patients have the slowest progression, while Alzheimer's disease (AD) patients have the fastest.
  - $a_i = 1$ (for CN patients)
  - $a_i = 2$ (for LMCI patients)
  - $a_i = 4$ (for AD patients)
- **Parameter b_i (Time Offset)**: This parameter is set to a random value, with the goal that at the patient's **first** observation time $t_{i,0}$, the transformed value $s = at + b$ falls within

a predefined s-value interval corresponding to that stage. This ensures a reasonable initial distribution of patient data points along the s-axis.

- For CN patients, $s_{i,0} \in (-10, 5)$
- For LMCI patients, $s_{i,0} \in (0, 10)$
- For AD patients, $s_{i,0} \in (5, 20)$

Through this step, we convert discrete clinical labels into a continuous yet meaningful initial distribution for the Disease Progression Score s.

2.2.2 Step 2: Fitting Population Data Trends with a Sigmoid Function

After obtaining the distribution of all patient data points in the (s, y) space, we use a flexible Sigmoid function to capture the macroscopic trend of each of the four biomarkers as they change with disease progression s. The Sigmoid model is well-suited for simulating the common "S"-shaped curves seen in biological processes.

For each biomarker k (where $k \in \{A_\beta, \tau, N, C\}$), we fit the following four-parameter Sigmoid function:

$$g_k(s; \theta_k) = \frac{p_1}{1 + e^{-p_2(s-p_3)}} + p_4 \tag{5}$$

Here, the parameter vector $\theta_k = (p_1, p_2, p_3, p_4)$ controls the amplitude, slope, center, and vertical offset of the curve, respectively. This step provides us with a smooth, noise-free, idealized trajectory for the biomarkers.

2.2.3 Step 3: Fitting the Polynomial Model Based on Sigmoid Derivatives

This step is the core of the pre-training phase. Instead of having the polynomial model $p(y)$ directly fit the discrete data points, we have it learn and approximate the **derivatives** of the smooth Sigmoid curves obtained in the previous step. This greatly improves the stability and accuracy of the fit.

The algorithm is as follows:

1. **Create a Grid**: We create a dense grid of points, s_{grid}, over the main interval of the Disease Progression Score $[-10, 20]$.
2. **Calculate Sigmoid Values and Derivatives**:
   - Using the fitted Sigmoid function $g_k(s)$ from Step 2, we calculate the biomarker values $y_j = (g_A(s_j), g_T(s_j), g_N(s_j), g_C(s_j))$ at each grid point s_j.
   - Concurrently, we compute the **analytical derivative** of the Sigmoid function, $\frac{\mathrm{d}g_k}{\mathrm{d}s}$, to get the derivative value $\frac{\mathrm{d}y_k}{\mathrm{d}s_j}$ at each grid point. This derivative represents the ideal rate of change of the biomarkers at any stage of the disease.
3. **Construct a Linear System**: Our goal is to find the coefficients w of the polynomial model such that $p(y_k; w) \approx \frac{\mathrm{d}y_k}{\mathrm{d}s}$. For each biomarker's differential equation (e.g., $\frac{\mathrm{d}A}{\mathrm{d}s} = w_{A,0} + w_{A,1}A + w_{A,2}A^2$), this can be formulated as a classic linear system $\Phi w = b$:
   - **Feature Matrix $\boldsymbol{\Phi}$**: Each row of the matrix corresponds to the state y_k at a grid point s_j and is constructed from the polynomial basis functions (e.g., $1, A, A^2$).
   - **Target Vector $\boldsymbol{b}$**: Each element of the vector is the corresponding Sigmoid derivative value $\frac{\mathrm{d}y_k}{\mathrm{d}s_j}$ at that grid point.
4. **Solve for Coefficients**: We can efficiently and accurately solve for the optimal polynomial coefficients w using standard **Linear Least Squares**.

For the polynomial model, we simplify by following the conclusions of Zheng et al[1]., employing a quadratic polynomial model where the couplings adhere to the transmission chain $A_\beta \to \tau \to N \to C$, thereby discarding non-adjacent couplings such as $A_\beta - N$ and $\tau - C$.

Through this process, we obtain a polynomial model $p(y)$ that can accurately reproduce the macroscopic dynamic rates of change of the biomarkers, along with a corresponding set of calibrated DPS parameters (a_i, b_i). These results are saved and serve as a stable starting point for the training of the hybrid model in the next stage.

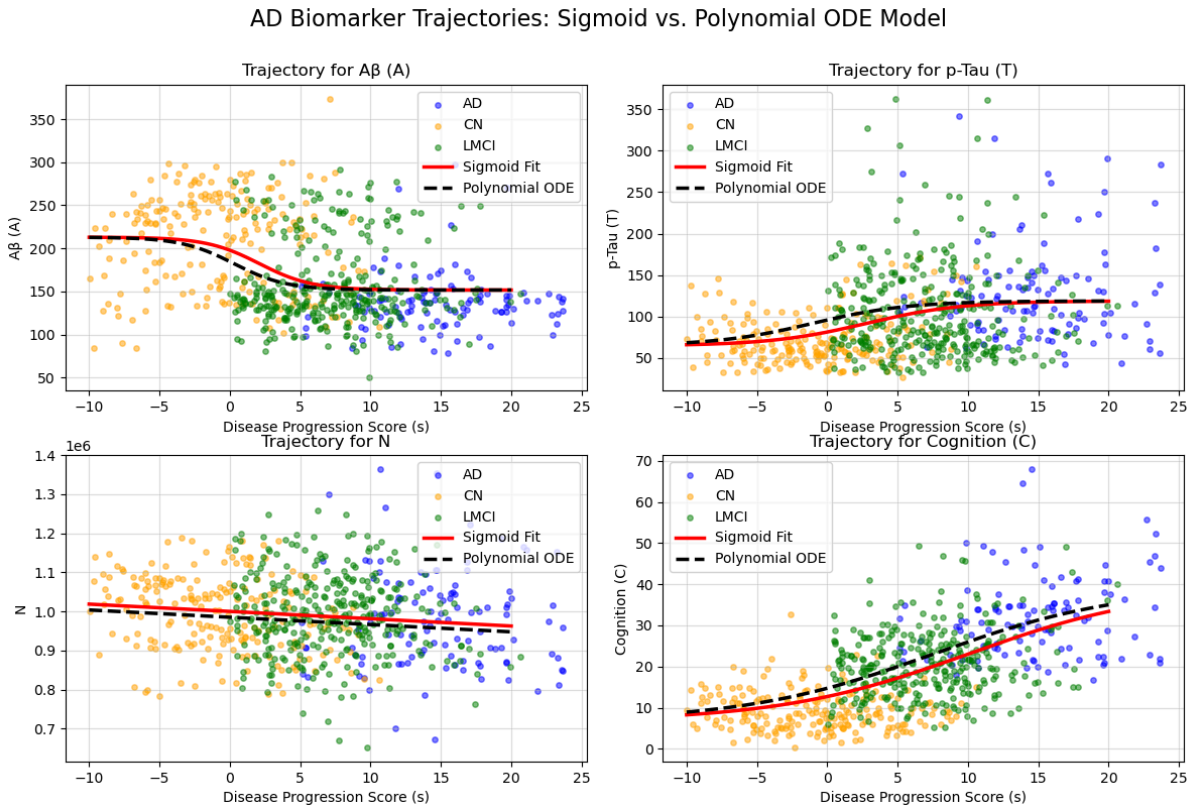

Figure 2: Data distribution and Polynomial ODE Trajectory after the DPS transformation from the pre-training process.

3 Population Model

Now we start introduce the main model and main algorithm.

3.1 Training Algorithm

For the time being, we make no assumptions regarding the specific form of the right-hand side in the equations, and denote it simply as f. Starting from initial condition $s = -10, y_0$, where y_0 be mean value of first observation of CN patients, we apply the 4-order Runge–Kutta method to iteratively compute the values of y at each grid point of s. We then calculate the mean squared error (MSE) between these results and the population data points (after filtering out outliers). Here s grid is all s values of our filtered data. The 4-order Runge-Kutta method, RK4, could be described as follows:

$$
\begin{aligned}
k_1 &= f(y_i, s_i) \\
k_2 &= f\left(y_i + \frac{h}{2}k_1, s_i + \frac{h}{2}\right) \\
k_3 &= f\left(y_i + \frac{h}{2}k_2, s_i + \frac{h}{2}\right) \\
k_4 &= f(y_i + hk_3, s_i + h) \\
y_{i+1} &= y_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}
\tag{6}
$$

The specific mathematical expression of the MSE loss is as follows:

$$
L = \frac{1}{4}\sum_{k=1}^{4} \frac{1}{\sum_{i=1}^{I} J_i} \sum_{i=1}^{I} \sum_{j_i=1}^{J_i} \left[y_{i,j,k} - \tilde{y}_{i,j,k}\right]^2 \tag{7}
$$

here I denotes the number of valid patients and J_i denotes number of time points of patient i's data. $\tilde{y}_{i,j,k}$ denotes the prediction of patient i's k-th biomarker at j-th time point.

3.2 Hybrid Model Training Stage

With the stable, pre-trained polynomial model $p(y)$ and DPS parameters (a_i, b_i) in hand, we proceed to train the full hybrid model. The core of this stage is to learn the neural network component $f(y)$, which acts as a correction term to capture the fine-grained, complex dynamics that the polynomial model alone cannot represent.

Since standalone FNN and polynomial model are both limited in terms of accuracy and interpretability, our primary focus is on integrating them:

$$
\frac{\mathrm{d}y}{\mathrm{d}s} = f(y) + P(y) \in R^4 \tag{8}
$$

This can be interpreted as a form of residual learning, where the quadratic polynomial serves as the interpretable backbone and the FNN acts as a residual component that supplements the dynamic terms. Accordingly, in the trajectory–scatter plots, we additionally plot the trajectories predicted by each of the two models individually as references.

Meanwhile, we try apply a network with one wide(1024 neurons) hidden layer, which shows a higher non-linear approximation ability compared with normal FNNs and polynomial model.

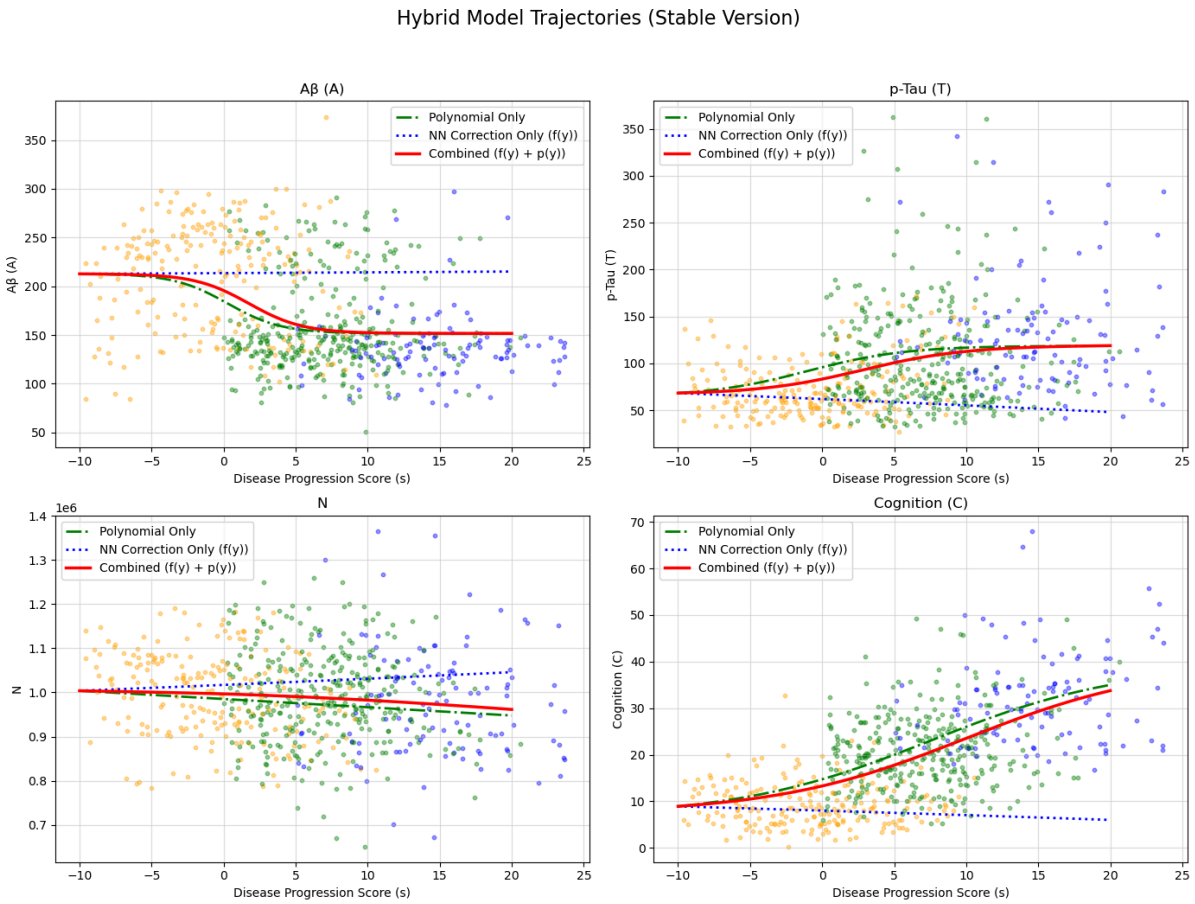

Figure 3: FNN + P Model Population Model Evaluation

To manage the complexity of optimizing different sets of parameters that have varying sensitivities and scales, we have designed an alternating optimization algorithm. This algorithm iteratively optimizes two distinct groups of parameters in each epoch: the neural network model's parameters, which is more robust, and fine tune the DPS parameters, as well as polynomial coefficients, which are more sensitive.

It is worth noting that, due to the numerical stability issues of RK4 integration, even slightly larger outputs from the neural network can cause the trajectories to diverge explosively. To mitigate this, we adopt the following initialization strategy: weights are initialized from a normal distribution with mean 0 and variance 0.001, while biases are initialized to 0. This may result in trajectories remaining close to linear, but it is still preferable to numerical explosion.

3.3 Uncertainty Quantification

We quantify the model's uncertainty using a random sampling method. Specifically, we assume the parameters follow a normal distribution, $\mathcal{N}(\mu, \sigma^2)$, where μ equals to the parameters from our trained model and $\sigma = 0.1$. We draw 100 random samples to obtain 100 different models. To characterize the possible range of the trajectories, we plot the 2.5th and 97.5th percentiles of the set of trajectories as the lower and upper bounds of the confidence interval.

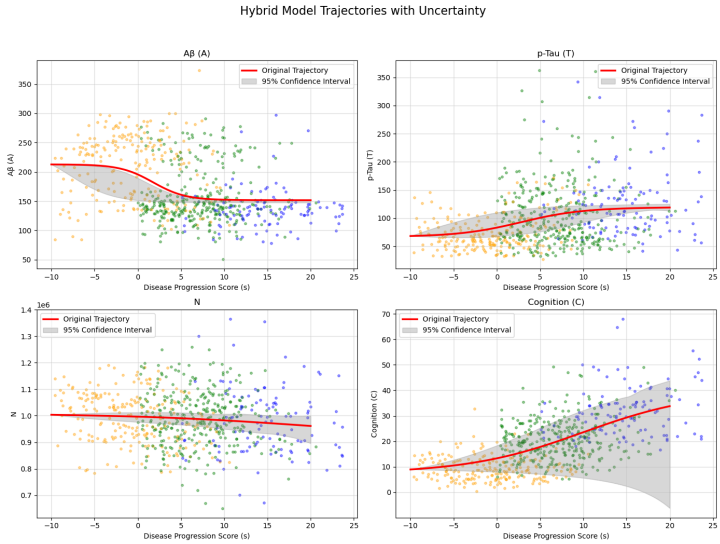

Figure 4: Hybrid Model Uncertainty Quantification

4 Personalized Model Construction

We perform personalized modeling based on parameter sensitivity analysis and transfer learning. The specific pipeline is as follows: first, we analyze the sensitivity of all parameters in the model (excluding dps parameters) to identify the most important 2% of parameters. Then, for each sample (patient), we fine-tune the population model with respect to these key parameters.

4.1 Sensitivity Analysis

The most accurate sensitivity analysis is variance-based sensitivity analysis. However, this method is computationally very expensive and is suitable for smaller-scale models. For example, this method is applicable when using only the polynomial model. But neural networks have at least thousands of parameters, and applying Sobol sensitivity analysis to a neural network would result in a computational cost of at least 1TB or more. Therefore, we adopt a simpler sensitivity analysis method for neural networks: (Introduce gradient-based sensitivity analysis and ranking methods).

4.2 Fine-tuning

The fine-tuning strategy is as follows: for each individual case, we select their first 3 observations. The model starts from the true initial value $y_{i,0}$ and predicts the values at all time points (observations). The L2 loss is calculated for the first 3 observations and the parameters are updated. Subsequent observations are reserved for cross-validation. A very small learning rate (e.g., 1e-7) is used when updating the parameters.

$$L_i = \frac{1}{4} \sum_{k=1}^{4} \frac{1}{\sum_{i=1}^{I} J_i} \sum_{j_i=1}^{J_i} \left[y_{i,j,k} - \tilde{y}_{i,j,k} \right]^2 \tag{9}$$

Therefore, we randomly select 4 samples from those with at least 4 observations and plot the prediction graphs to evaluate the effectiveness of this modeling method. Through Figure 5, we could find that the personalized model performs better than the populational model in terms of training data and prediction data.

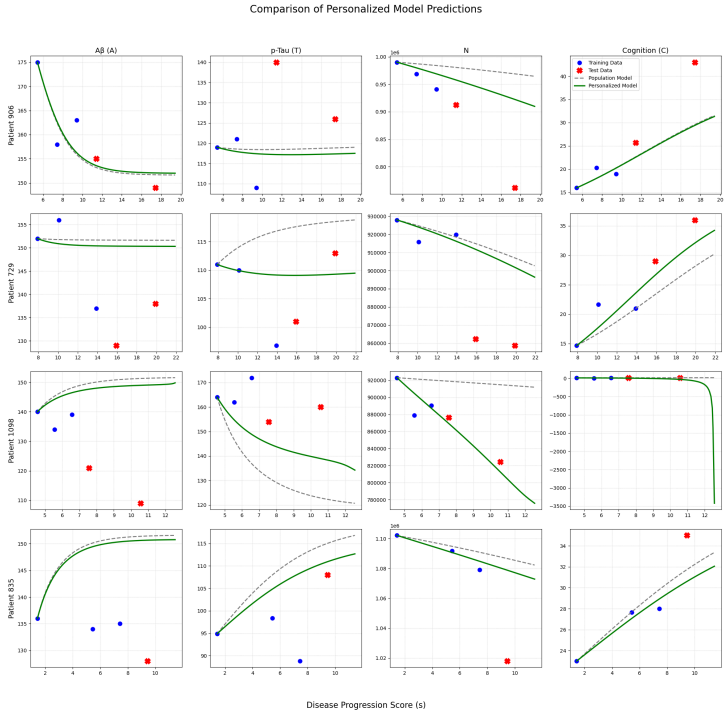

Figure 5: Personalized Model Evaluation

Bibliography

[1] H. Zheng, J. R. Petrella, P. M. Doraiswamy, and others, "Data-driven causal model discovery and personalized prediction in Alzheimer's disease," *npj Digital Medicine*, vol. 5, no. 1, p. 137, 2022, doi: 10.1038/s41746-022-00632-7.